

# PRD — Find Your Train

## 1. Product Overview

**Website Name:** Find Your Train

**Purpose:**

A responsive frontend prototype for dynamically displaying train location, ETA, delay, route and operating conditions.

The frontend will be integrated later with a backend containing the CatBoost ETA prediction model.

### Frontend Scope

The frontend must provide:

- Passenger interface
- Admin interface
- Interactive India map using MapTiler
- Train route and station visualization
- Live/simulated train location
- ETA and delay display
- Weather, temperature, speed and congestion display
- Admin control of operating conditions
- 5 km checkpoints for admin
- Real-time reflection of admin changes on passenger view
- Dark/light mode
- Responsive design
- Mock data mode for demonstration

### Not in frontend scope

Do not implement:

- CatBoost model
- ML calculations
- Backend/database
- GPS ingestion
- Railway control logic
- Real authentication system

---

## 2. Prototype Trains

Support these four trains:

## **Express**

**12301 — Howrah–New Delhi Rajdhani Express**

## **Local**

**36835 — Howrah–Bardhaman Chord Local**

**63296 — DDU–Gaya MEMU**

**64423 — Ghaziabad–New Delhi EMU**

Use reusable train components so more trains can be added later.

---

# **3. Entry Screen**

When the website opens, show:

## **Find Your Train**

### **Dynamic Railway ETA & Journey Tracking**

Two large options:

#### **Continue as Passenger**

Track train location, ETA, route and conditions.

#### **Continue as Admin**

Monitor trains and modify operating conditions.

Also provide a **Light/Dark mode toggle**.

---

# **4. Passenger Interface**

## **Train Selection**

Display the four trains as cards.

Each card shows:

- Train number
- Train name
- Express/Local

- Source → Destination
- Current status
- Current delay

Selecting a train opens its dashboard.

---

## Passenger Dashboard

Main dashboard should contain:

### Train Header

- Train name
- Train number
- Running status
- Current delay
- Last updated time

### ETA Card

Show:

- Expected arrival
- Scheduled arrival
- Predicted delay

The ETA must come from the backend/ML model.

**Frontend must never calculate the ETA.**

### Journey Progress

Show:

- Journey percentage
- Distance travelled
- Distance remaining
- Current station/segment
- Next station

### Interactive India Map

Use **MapTiler**.

Map should display:

- Complete railway route
- Completed route
- Remaining route
- Train location
- Stations

- Current station
- Next station
- Passed stations

Station states:

Upcoming  
Current  
Passed  
Destination

When a station is crossed, automatically mark it as **Passed** based on backend train progress.

Clicking a station should show basic information such as ETA and delay.

## Conditions

Show:

- Speed
- Weather
- Temperature
- Congestion

Example:

Speed: 72 km/h  
Weather: Rainy  
Temperature: 27°C  
Congestion: High

Passengers can view these values but cannot edit them.

---

# 5. Admin Interface

## Admin Dashboard

Show all four trains.

Each train card should show:

- Speed
- Weather
- Temperature
- Congestion
- Predicted delay

- Current status

Button:

## Control Train

---

## Admin Train Control

Admin can modify:

### Speed

Slider/numeric input.

### Weather

Options:

Sunny  
Cloudy  
Rainy  
Storm  
Fog  
Clear

### Temperature

Numeric input.

### Congestion

Low  
Moderate  
High  
Severe

Optional operational controls:

Speed Restriction  
Signal Halt  
Unscheduled Stoppage  
Maintenance Block

Button:

### Apply Changes

After applying changes:

1. Frontend updates the current state.
2. Frontend sends changes to backend.
3. Backend/ML model generates updated ETA.
4. Passenger dashboard receives updated information.

---

## 6. Checkpoint System

Create checkpoints every **5 km along the railway route**.

Checkpoints are **ADMIN ONLY**.

Passengers must never see checkpoint controls.

### Admin Map

Show:

CP-001  
CP-002  
CP-003  
...

Each checkpoint can show:

- Checkpoint ID
- Distance
- Status
- Current conditions

Admin can select:

### Activate Checkpoint

The checkpoint utility should calculate points along the actual route geometry rather than simply connecting stations with straight lines.

Use GeoJSON route data and Turf.js or allow the backend to provide checkpoint coordinates.

---

## 7. Real-Time / Mock Mode

The frontend must work without the backend.

Use:

```
VITE_USE MOCK_API=true
VITE_API_BASE_URL=http://localhost:8000/api
VITE_MAPTILER_API_KEY=
```

Mock mode should simulate:

- Train movement
- ETA changes
- Station crossing
- Condition changes
- Admin → Passenger updates

For example:

```
Admin changes:
Congestion: Moderate → High
Speed: 82 → 55 km/h
```

↓

```
Passenger sees:
Congestion: High
Speed: 55 km/h
Predicted delay: updated
```

Use an abstraction so mock mode can later be replaced by REST/WebSocket/SSE.

---

## 8. Frontend Architecture

Recommended stack:

- React
- TypeScript
- Vite
- Tailwind CSS
- React Router
- Zustand
- MapTiler SDK
- Turf.js

Suggested structure:

```
src/
├─ components/
│   └─ passenger/
│   └─ admin/
```

```
|   ├── map/
|   └── common/
├── pages/
├── services/
├── store/
├── mock/
├── types/
├── utils/
└── App.tsx
```

Create reusable components such as:

```
TrainCard
TrainSelector
ETACard
JourneyProgress
WeatherCard
ConditionsCard
TrainMap
StationMarker
TrainMarker
CheckpointMarker
AdminControlPanel
```

Keep API calls inside `services/`, not directly inside UI components.

---

## 9. Basic API Contract

The frontend should be prepared for APIs such as:

```
GET  /api/trains
GET  /api/trains/:trainNumber
GET  /api/trains/:trainNumber/route
GET  /api/trains/:trainNumber/live
GET  /api/trains/:trainNumber/prediction
GET  /api/trains/:trainNumber/conditions

PATCH /api/admin/trains/:trainNumber/conditions

POST  /api/admin/trains/:trainNumber/checkpoints/:checkpointId/activate
```

The frontend should consume backend-provided:

```
train location
speed
```

delay  
ETA  
stations  
route geometry  
weather  
temperature  
congestion

---

## 10. Design Requirements

### Visual Style

Use a modern railway/transport dashboard style.

Primary colors:

Blue  
White  
Navy  
Light gray

Use status colors for:

On Time  
Warning  
Delayed  
Critical

### Dark Mode

Provide a global light/dark toggle.

The MapTiler map should also switch between light and dark styles.

### Responsive

Must work well on:

- Mobile
- Tablet
- Laptop
- Desktop

Passenger mobile order:

Train Info  
ETA  
Next Station  
Map  
Journey Progress  
Weather  
Conditions  
Upcoming Stations

---

## 11. Important Demo Flow

The main SIH demonstration should be:

```
Open Find Your Train
  ↓
Continue as Admin
  ↓
Select 12301
  ↓
Change Speed / Weather / Congestion
  ↓
Apply Changes
  ↓
Backend / Mock ML updates ETA
  ↓
Open Passenger
  ↓
Select 12301
  ↓
Passenger sees updated conditions + ETA
```

This should be the primary interaction the prototype is designed to demonstrate.

---

## 12. External APIs

### MapTiler — Required

Use it for:

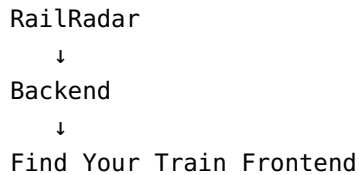
- India map
- Railway route
- Train location
- Station markers
- Route visualization

## RailRadar — Optional

Not required for the frontend prototype.

It can be integrated later through the backend if the team wants real train location, route or station data.

Recommended architecture:



Do not make the frontend directly dependent on RailRadar.

## Weather API — Not Required

For this prototype, weather and temperature are controlled by the admin, so a separate weather API is unnecessary.

---

# 13. Acceptance Criteria

The frontend is complete when:

- Passenger/Admin entry works.
- All four trains are available.
- Passenger can select a train.
- Interactive MapTiler map works.
- Route and stations are displayed.
- Train position is displayed.
- Passed stations update automatically.
- ETA and predicted delay are displayed.
- Weather, temperature, speed and congestion are displayed.
- Admin can modify conditions.
- Admin-only checkpoints work.
- Passenger cannot access checkpoint controls.
- Admin changes appear in Passenger mode.
- Mock mode works without a backend.
- Backend API integration can be added without rewriting the UI.
- Light/dark mode works.
- Mobile and desktop layouts are responsive.

## Final Principle

The frontend should visualize:

**Train State → Operating Conditions → ML Prediction → Passenger ETA**

The frontend should **display and transmit data**, not perform the ML prediction itself.